UAV Research Onboarding

Linux, Git, ROS 2, PX4 and Gazebo, for master's thesis students at Politecnico di Torino

Riccardo Enrico

Abstract

This guide teaches the minimum you need to start work in a UAV research group. It assumes you have never used Linux, Git, ROS 2, PX4 or Gazebo. It covers only the tools a typical group uses: Ubuntu, ROS 2, PX4 firmware with Gazebo simulation, Isaac Sim, and a ground station. Read it once, then use the First-day checklist on your first day.

Contents

1 Versions **2**

2 Linux **2**
2.1 The terminal and the filesystem . . . 2
2.2 Permissions and sudo . . . 3
2.3 Environment variables and source . . . 3
2.4 Installing software with apt . . . 4
2.5 Processes . . . 4
2.6 SSH into the onboard computer . . . 4
2.7 Editors . . . 4

3 Git and GitHub **4**
3.1 Daily commands . . . 5
3.2 Branches . . . 5
3.3 Commit messages . . . 6
3.4 Issues and pull requests . . . 6
3.5 Submodules . . . 6

4 ROS 2 **7**
4.1 Installing ROS 2 Jazzy . . . 7
4.2 Installing the other tools . . . 8
4.3 Concepts . . . 8
4.4 QoS: why PX4 topics need special settings . . . 8
4.5 Workspaces and colcon . . . 9
4.6 Command line tools . . . 9
4.7 A minimal node . . . 10

5 PX4 **11**
5.1 Architecture . . . 11
5.2 Flight modes and offboard mode . . . 11
5.3 The uXRCE-DDS bridge . . . 13
5.4 Frames: NED/FRD versus ENU/FLU . . . 13
5.5 Simulation: SITL with Gazebo . . . 14
5.6 QGroundControl, parameters and safety . . . 14

6 Isaac Sim **17**

7 How it fits together **17**
7.1 First-day checklist 18

8 Further reading **18**

1 Versions

Everything below targets these versions. When the group upgrades, change this table and check the commands that mention a version.

Component	Version	Note
Ubuntu	24.04	the only supported Linux; do not install 26.04, even though it is the newest LTS: ROS 2 Jazzy targets 24.04. Check with `lsb_release -a`
ROS 2	Jazzy	docs: docs.ros.org/en/jazzy
PX4 firmware	1.17	docs: use the page for 1.17
`px4_msgs`	branch `release/1.17`	must match the firmware
Gazebo	Harmonic (`gz`)	as installed by the PX4 setup script
Micro XRCE-DDS Agent	as installed in the lab	`MicroXRCEAgent`
QGroundControl	current stable	ground station
Isaac Sim	ask your supervisor	see Isaac Sim

2 Linux

2.1 The terminal and the filesystem

Almost all work happens in a terminal. You type a command, press Enter, and read the output. The shell (here `bash` or `zsh`) is the program that reads your commands.

Linux has one directory tree that starts at `/` (the root). Your own files live in your home directory, written `~` (for example `/home/riccardo`). A *path* says where a file is. An *absolute* path starts with `/` or `~`. A *relative* path starts from the current directory. The path `.` is the current directory and `..` is its parent.

```
pwd                       # print the current directory
ls -l                     # list files, one per line, with details
cd ~/my_repo              # change directory
cd ..                     # go to the parent directory
mkdir -p ws/src           # make a directory (and parents)
cp a.txt b.txt            # copy;   mv a.txt c.txt  moves or renames
rm b.txt                  # delete a file (no recycle bin!)
cat file.txt              # print a file;  less file.txt  to scroll
grep -rn "offboard" src/  # search text in files
man ls                    # manual page of a command (q to quit)
```

Tips: press Tab to complete names. Press the up arrow to recall old commands. Press Ctrl-R to search your history. File names are case sensitive.

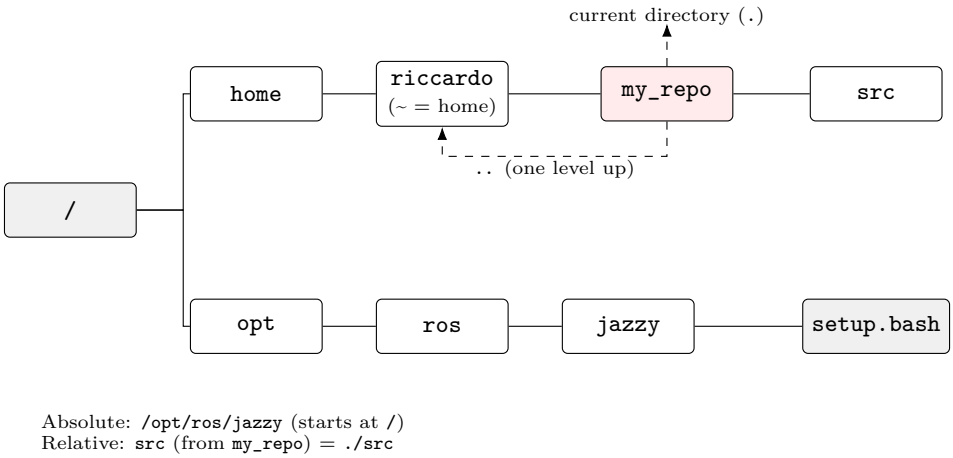


Figure 1: One directory tree, rooted at `/`. `~` is your home directory, `.` the current directory and `..` its parent.

2.2 Permissions and sudo

Each file has an owner and three permission sets: owner, group, others. Each set allows read (`r`), write (`w`) and execute (`x`). `ls -l` shows them, for example `-rwxr-xr-x`. A script must have the execute bit to run as `./script.sh`.

```
chmod +x script.sh        # make a file executable
sudo apt update           # run one command as administrator
```

Use `sudo` only when a command needs it (installing system packages). Never run ROS or your own code with `sudo`.

2.3 Environment variables and source

An environment variable is a named string that programs can read. ROS 2 depends on them. Print one with `echo $ROS_DISTRO`. Set one for the current shell with `export ROS_DOMAIN_ID=7`.

A setup file such as `/opt/ros/jazzy/setup.bash` sets many variables at once (it extends `PATH`, the list of folders where the shell looks for programs). You must *source* it, not run it:

```
source /opt/ros/jazzy/setup.bash
```

Sourcing changes only the current terminal. When you open a new terminal you must source again, or put the line in `~/.bashrc` (read at the start of every bash terminal). A frequent beginner error is "command not found" for `ros2` or `colcon`: you forgot to source.

Note

If your shell is `zsh`, source the `.zsh` file instead (`/opt/ros/jazzy/setup.zsh`) and put the line in `~/.zshrc`.

Tip

Put the base ROS 2 setup file (`setup.bash` or `setup.zsh`) in your shell startup file. Source the overlay of your workspace (`install/setup.bash`) only when you work in that workspace (Workspaces and colcon).

2.4 Installing software with apt

Ubuntu installs system software with `apt`.

```
sudo apt update                          # refresh the package list
sudo apt install git build-essential     # install packages
apt search ros-jazzy-rqt                 # search
```

2.5 Processes

Every running program is a process. Press Ctrl-C in a terminal to stop the program in it. Ctrl-Z suspends it. If a program hangs, or a simulator stays alive in the background:

```
ps aux | grep -i gz        # find processes whose line contains "gz"
kill 12345                 # ask process 12345 to stop (12345 is the PID)
kill -9 12345              # force it, if the first kill did not work
pkill -f px4               # kill by name pattern
htop                       # live process viewer (q to quit)
```

You will run several programs at once (the simulator, the bridge to PX4, your own nodes). Use one terminal for each, or a terminal multiplexer such as `tmux`.

2.6 SSH into the onboard computer

The drone carries a small computer. You log in to it over the network with SSH.

> **Note**
>
> The lab drones are already set up. Read this section for background, and use it when you set up new hardware.

```
ssh-keygen -t ed25519              # once: make a key pair
ssh-copy-id user@drone-ip          # copy the public key to the drone
ssh user@drone-ip                  # log in; type "exit" to leave
scp file.txt user@drone-ip:~/      # copy a file to the drone
```

Ask your supervisor for the user name and the IP address of the onboard computer; they are not written here on purpose. The same key type is used to push to GitHub (Git and GitHub).

2.7 Editors

Use what you like. Recommended editors:

- VS Code: the most used in the group. Its "Remote - SSH" extension edits files on the drone.
- Zed: built in Rust, faster.
- Neovim: not beginner friendly.

`nano file` is the simplest terminal editor (Ctrl-O saves, Ctrl-X exits). If you land in `vim` by accident, press Esc, then type `:q!` and Enter to leave.

3 Git and GitHub

Git records the history of a folder (a *repository*). Each saved state is a *commit*. GitHub hosts a copy of the repository online and adds issues and pull requests. Lab repositories are usually private. Ask for access, add your SSH public key (`~/.ssh/id_ed25519.pub`) in your GitHub account settings, and test with `ssh -T git@github.com`.

3.1 Daily commands

```
git clone --recursive git@github.com:<owner>/<repo>.git
cd <repo>
git status                  # what changed?
git diff                    # show the changes in detail
git add src/my_node.cpp     # stage a file for the next commit
git commit -m "feat(controller): add acceleration limit"
git push                    # upload your commits
git pull                    # download and merge new commits
git log --oneline -10       # last 10 commits
```

The flow is: edit files, `git add` the ones that belong together, `git commit` with a message, `git push`. A commit is local until you push.

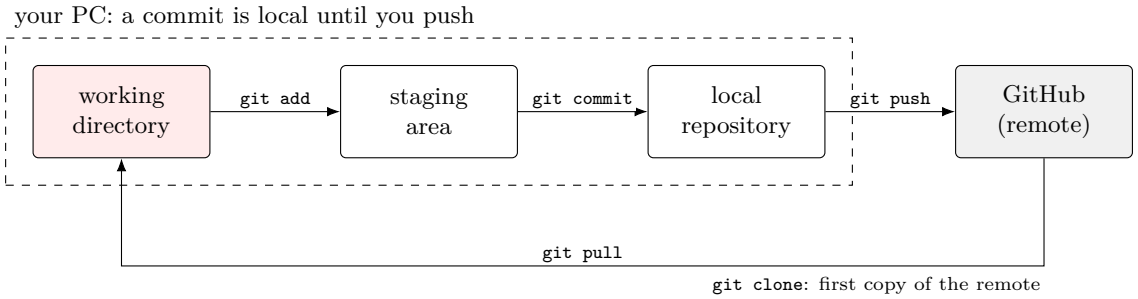


Figure 2: `git add` stages, `git commit` records locally, and only `git push` sends the commits to GitHub. `git pull` brings new commits back.

Write commit messages as Conventional Commits: `<type>(<scope>): <description>`, for example `feat(controller): add acceleration limit` or `fix(bridge): correct frame conversion`.

3.2 Branches

A branch is a movable label for a line of commits. Do your work on a branch, not on `main`.

```
git switch -c feat/12-hover-node       # make and switch to a new branch
git switch main                        # go back
git push -u origin feat/12-hover-node  # first push of a new branch
```

Branch names start with the type (`feat`, `fix`, `docs`, `chore`) and should contain the issue number.

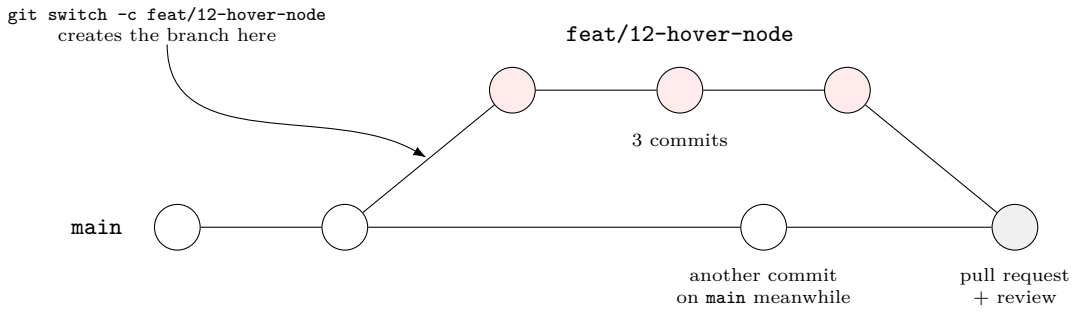


Figure 3: `git switch -c` starts a branch off `main`. You commit on it, and a reviewed pull request merges it back.

3.3 Commit messages

Messages follow Conventional Commits: `type(scope): description`. Examples:

```
feat(controller): add tilt limit
fix(mapper): clamp negative distance
docs(guide): add ROS 2 section
```

Use the imperative, lower case, no final period. Add `refs #12` (or `fixes #12`) at the end of the body to link the commit to an issue.

3.4 Issues and pull requests

An *issue* describes one task. Check the open issues before you start, and write one for new work. A *pull request* (PR) asks to merge your branch into `main`. Someone reviews it, then it is merged. If `git pull` reports a *conflict*, Git marks the clashing lines with `<<<<<<<`; edit the file to keep what you want, `git add` it, and `git commit`.

3.5 Submodules

A project can include another repository inside it. A typical case in a PX4 project is the message package:

Path	Content	Pinned to
`src/px4_msgs`	PX4 message definitions (public)	branch `release/1.17` (e.g. tag `v1.17.0`)

A submodule is pinned to one commit. The parent repo does not store the files of the submodule. It stores the URL (in `.gitmodules`) and one exact commit hash. When you clone, you get that commit and nothing else, so everyone builds the same code.

```
# add it once, following the firmware's release branch
git submodule add -b release/1.17 \
  https://github.com/PX4/px4_msgs.git src/px4_msgs

# fresh clone with everything, including nested submodules
git clone --recursive git@github.com:<owner>/<repo>.git

# you cloned without --recursive, or someone bumped a submodule:
git submodule update --init --recursive

git submodule status        # shows the pinned hash of each one
```

A leading `+` in the status means the submodule folder is at a different commit than the parent pins. A leading `-` means it is not initialised yet. After a plain `git pull` in the parent, run `git submodule update --init --recursive` so the folders follow the new pins.

Bumping a submodule. To move `px4_msgs` to a newer commit of its release branch:

```
cd src/px4_msgs
git fetch && git switch release/1.17 && git pull   # get the new commit
cd ../..
git status        # shows: modified: src/px4_msgs (new commits)
git add src/px4_msgs
git commit -m "build: bump px4_msgs"
```

Rule: if you change code *inside* a submodule you own, commit and push it *there first*, then commit the new pin in the parent. If you do it in the other order, other people cannot fetch the pinned commit. Do not bump `px4_msgs` on your own to a different release line: it must match the PX4 firmware (PX4).

4 ROS 2

ROS 2 is a set of libraries and tools to build robot software as many small programs that talk to each other. This guide uses Ubuntu 24.04 and ROS 2 **Jazzy** (see Versions). Use Jazzy commands and documentation (docs.ros.org/en/jazzy); older tutorials may differ.

4.1 Installing ROS 2 Jazzy

A new machine has no ROS 2. Check with `ls /opt/ros`: if the folder is missing or does not list `jazzy`, install the deb packages, following the official Ubuntu install page. Do the steps in order and check the output of each before the next.

1. Check that the locale supports UTF-8 (for example `en_US.UTF-8`). If it does not, the install page shows how to set one.

   ```
   locale
   ```

2. Enable the Ubuntu universe repository.

   ```
   sudo apt install software-properties-common
   sudo add-apt-repository universe
   ```

3. Add the ROS 2 apt repository. One package installs its keys and sources. Run these four lines in the same terminal: the second line sets a variable that the third line uses.

   ```
   sudo apt update && sudo apt install curl -y
   export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-
       infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F'"
       ' '{print $4}')
   curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-
       apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${
       ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo ${UBUNTU_CODENAME:-${
       VERSION_CODENAME}})_all.deb"
   sudo dpkg -i /tmp/ros2-apt-source.deb
   ```

4. Update the system. ROS 2 packages are built against an up-to-date Ubuntu.

   ```
   sudo apt update && sudo apt upgrade
   ```

5. Install ROS 2 and the build tools. `ros-jazzy-desktop` includes RViz, the demos and the tutorials; `ros-dev-tools` brings `colcon`. If `ros-dev-tools` fails with dependency conflicts, the install page has the fix.

   ```
   sudo apt install ros-jazzy-desktop ros-dev-tools
   ```

6. Test the install. In one terminal start a talker:

   ```
   source /opt/ros/jazzy/setup.bash
   ros2 run demo_nodes_cpp talker
   ```

   In a second terminal start a listener:

   ```
   source /opt/ros/jazzy/setup.bash
   ros2 run demo_nodes_py listener
   ```

   The talker prints `Publishing` lines and the listener prints `I heard` lines.

4.2 Installing the other tools

A new machine also needs these. Follow the official page for each:

- PX4-Autopilot and Gazebo: PX4 Ubuntu development environment. Its `ubuntu.sh` script installs the PX4 toolchain and Gazebo.
- Micro XRCE-DDS Agent: PX4 ROS 2 user guide, "Setup the Agent". It builds the agent inside your ROS 2 workspace.
- QGroundControl: QGC download and install.

4.3 Concepts

- A **node** is one program with one job (for example a mapper, or a controller).
- A **topic** is a named channel, such as `/camera/depth/points`. A node **publishes** messages to it; other nodes **subscribe** and receive them.
- A **message** is a typed record. Its type, like `px4_msgs/msg/TrajectorySetpoint`, lists named fields (`position`, `velocity`, `acceleration`, ...).
- A **parameter** is a named setting of a node (a gain, a horizon length) that you set at start or change at run time.
- A **launch file** (Python) starts many nodes with their parameters in one command.

Nodes do not know each other. They only know topic names and types. So a simulated sensor and a real sensor can be exchanged without changing the controller.

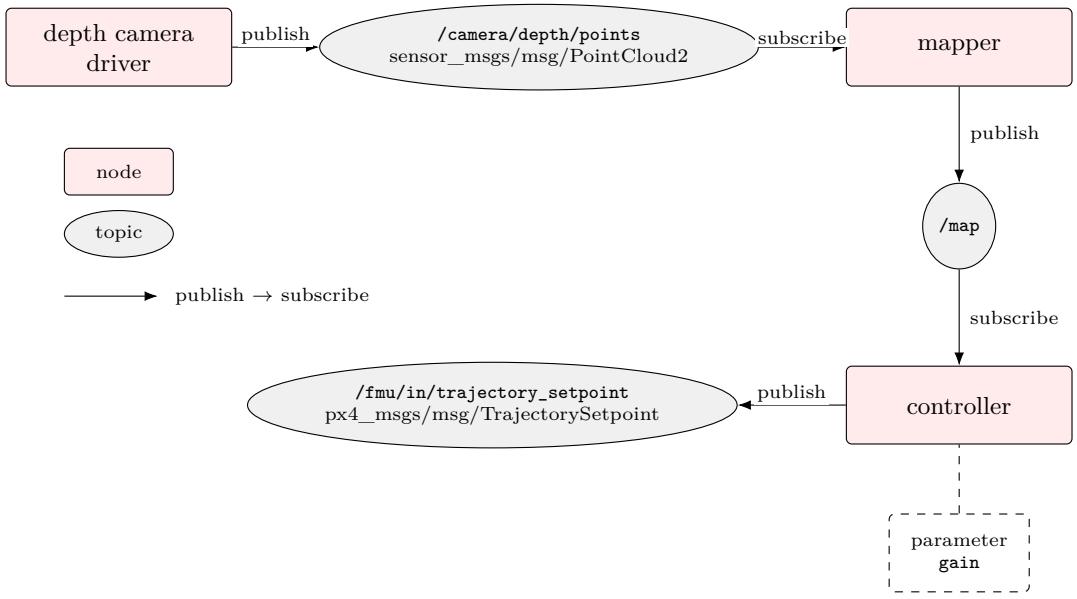


Figure 4: A ROS 2 graph. Nodes never call each other: they publish to and subscribe from named, typed topics. A node can expose parameters such as a gain.

4.4 QoS: why PX4 topics need special settings

Quality of Service (QoS) settings decide how messages are delivered. A publisher and a subscriber only connect if their settings are *compatible*. The two settings that matter here:

- **Reliability**: *reliable* resends lost messages; *best effort* does not. Sensor streams use best effort, since a late old sample is worse than a lost one.
- **Durability**: *volatile* sends only to subscribers present at publish time; *transient local* keeps the last message for late subscribers.

The ROS 2 default for a subscriber is reliable and volatile. PX4 does not use these defaults, so a default subscriber to a PX4 topic can silently receive nothing, which is the most common beginner failure. The PX4 documentation says subscribers "must specify a appropriate (compatible) QoS setting" and uses the predefined sensor-data profile (`rmw_qos_profile_sensor_data`: best effort, volatile, small queue) for PX4 output topics; its example sets a depth of 5. The guide gives no per-topic settings, so use that profile for every PX4 topic. Publishers to `/fmu/in/` need no special QoS: the ROS 2 default is compatible. If a subscription shows no data, check with `ros2 topic info -v <topic>`, which prints the QoS of each endpoint.

4.5 Workspaces and colcon

Your code lives in a *workspace*: a folder with a `src/` directory. `colcon` builds every package in `src/`.

```
source /opt/ros/jazzy/setup.bash
cd ~/my_repo
colcon build --packages-select px4_msgs     # build one package
source install/setup.bash                   # make the result visible
```

Colcon creates `build/`, `install/` and `log/`. Never edit them and never commit them. The order matters: first source the ROS base install (the *underlay*), then build, then source `install/setup.bash` (the *overlay*) in every new terminal where you want to run your packages. Build only what you changed with `--packages-select`; large packages can be slow to compile.

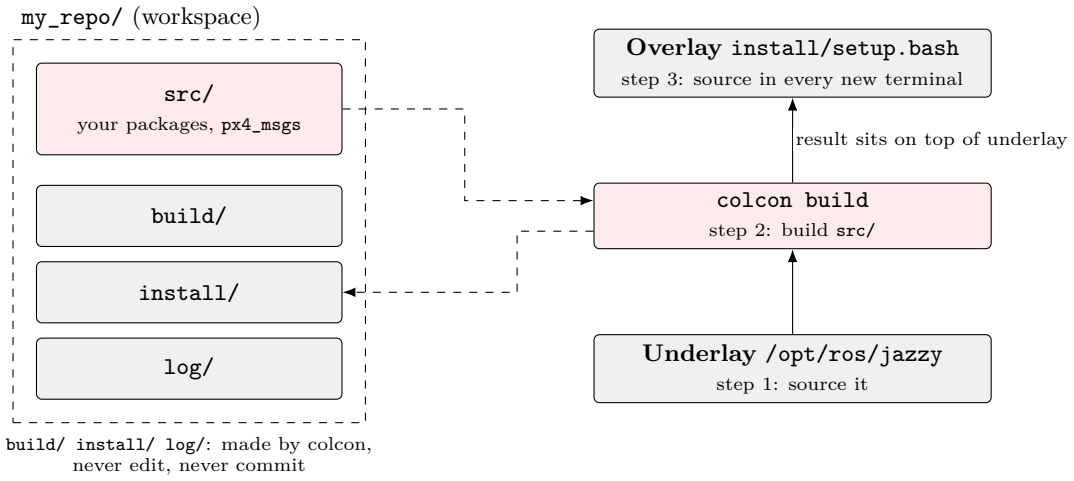


Figure 5: Your sources live in `src/`; colcon creates `build/`, `install/` and `log/`. Source the underlay, build, then source the overlay.

4.6 Command line tools

```
ros2 topic list                                 # all topics
ros2 topic info /fmu/out/vehicle_status -v      # type and QoS (name may differ)
ros2 topic echo <topic>                         # print messages
ros2 topic hz <topic>                           # publication rate
ros2 interface show px4_msgs/msg/TrajectorySetpoint   # message fields
ros2 node list                                  # running nodes
ros2 param list /my_node                        # parameters of a node
ros2 launch <package> <file>.launch.py          # start a launch file
rqt_graph                                       # picture of nodes and topics
rviz2                                           # 3D view: clouds, maps, paths
```

Before you use a message field, check its name with `ros2 interface show`. If two people on one network see each other's topics by accident, set a different `ROS_DOMAIN_ID` (0 to 101) in each terminal.

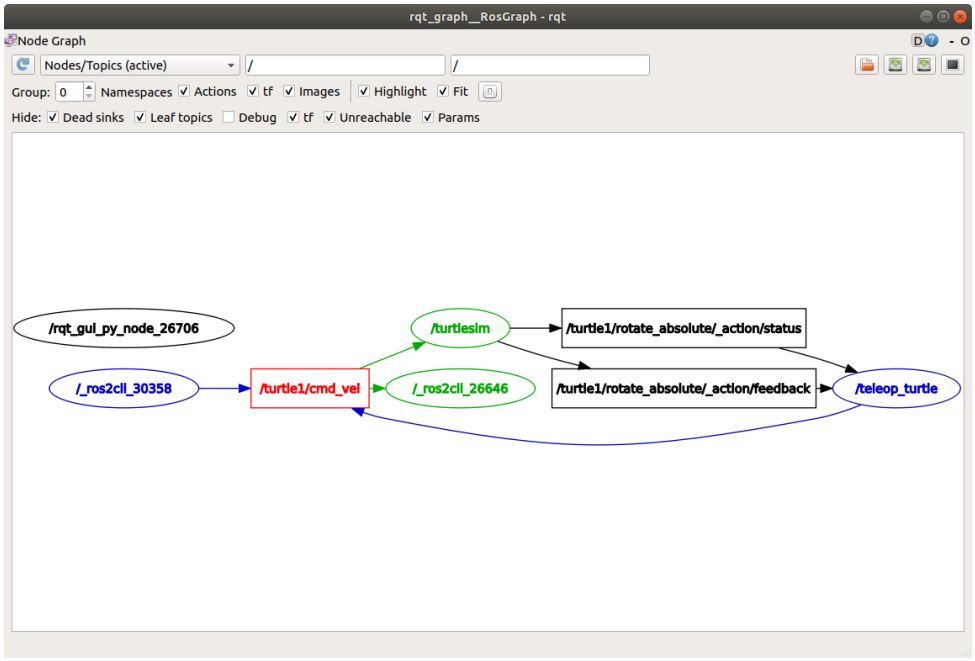


Figure 6: `rqt_graph` draws the live graph: here the turtlesim nodes and the `/turtle1/cmd_vel` topic. Image: ROS 2 Documentation (Jazzy), CC BY 4.0.

4.7 A minimal node

A subscriber in Python (`rclpy`) that prints the north position from PX4, with a compatible QoS. The field `x` exists in `px4_msgs/msg/VehicleLocalPosition`; it is the north position in the NED frame, in metres (checked in the `msg` folder of `px4_msgs`). The topic name must be checked with `ros2 topic list` (PX4).

```
import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from px4_msgs.msg import VehicleLocalPosition

class Echo(Node):
    def __init__(self):
        super().__init__("echo")
        self.create_subscription(
            VehicleLocalPosition, "/fmu/out/vehicle_local_position",
            self.cb, qos_profile_sensor_data)

    def cb(self, msg):
        self.get_logger().info(f"north = {msg.x:.2f} m")

rclpy.init()
rclpy.spin(Echo())
```

Run it with `python3 echo.py` in a terminal where both ROS and the workspace are sourced. C++ nodes (`rclcpp`) have the same structure: create the node, create subscriptions and publishers, spin.

5 PX4

5.1 Architecture

PX4 is the flight-controller firmware. It runs on the flight-controller board (or, in simulation, as a normal program on your PC). It reads the IMU and other sensors, estimates the vehicle state (with an EKF), and runs the control loops that turn setpoints into motor commands. Inside PX4, modules communicate through **uORB**, a publish/subscribe system like ROS topics but internal to the firmware. PX4 does not run ROS. A bridge (uXRCE-DDS bridge) copies chosen uORB topics to ROS 2.



Figure 7: A flight-controller board (Holybro Pixhawk 6C) with its ports. PX4 runs on it. Image: PX4 User Guide (Holybro), CC BY 4.0.

Control is hierarchical. Your controller outputs a setpoint, for example an acceleration. PX4's position and velocity controllers turn it into an attitude and thrust setpoint; the attitude and rate loops close the rest at a much higher rate. This is why a high-level planner can often treat the vehicle as a point mass: the fast inner loops are PX4's job.

5.2 Flight modes and offboard mode

A flight mode decides who produces the setpoints. Manual and stabilised modes follow the pilot's sticks. Position mode holds position. Hold, Return and Land are automatic. **Offboard** mode follows setpoints that arrive from an external computer, which is your ROS 2 node.

Rules for offboard mode, from the PX4 documentation:

- PX4 must *already receive* `OffboardControlMode` messages before it accepts offboard mode or arms in it. Start the stream first.
- `OffboardControlMode` must be sent together with a setpoint message (for example

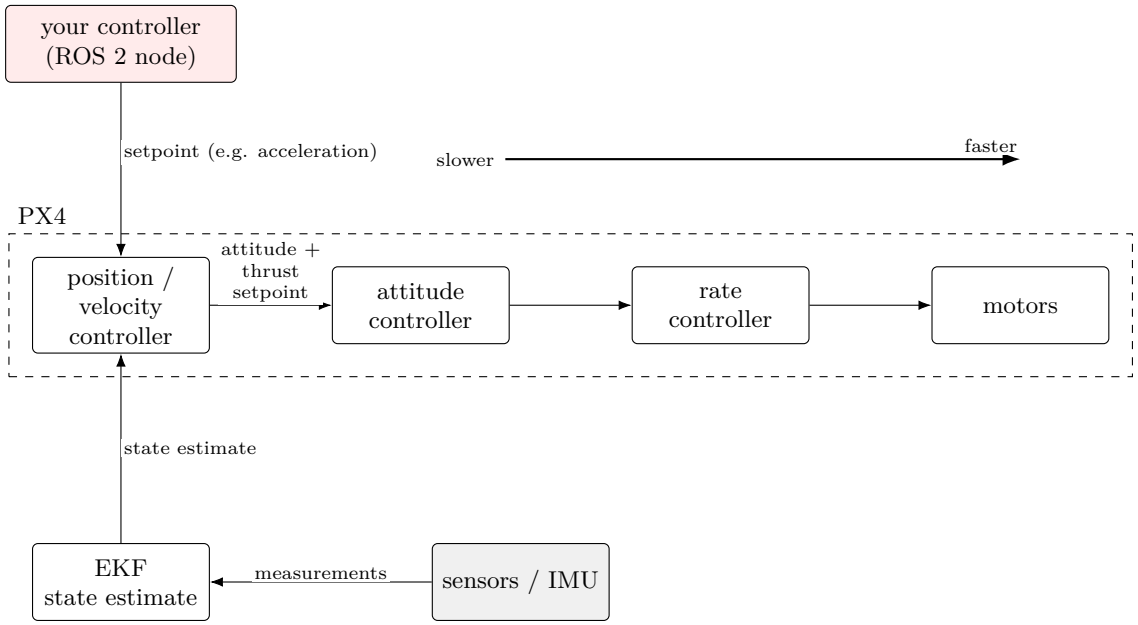


Figure 8: Control hierarchy. Your node sends a setpoint; PX4's position, attitude and rate loops, each faster than the last, turn it into motor commands. The EKF feeds the state estimate back to the position controller.

`TrajectorySetpoint`). The first message says *which* fields are used; the second carries the values.

- If the stream of `OffboardControlMode` drops below about 2 Hz, PX4 leaves offboard mode (failsafe). Send both at a steady rate well above that. The official example uses a 100 ms timer (10 Hz); 50 Hz or more is common.
- The official example sends 10 setpoints, then requests the mode switch, then arms.

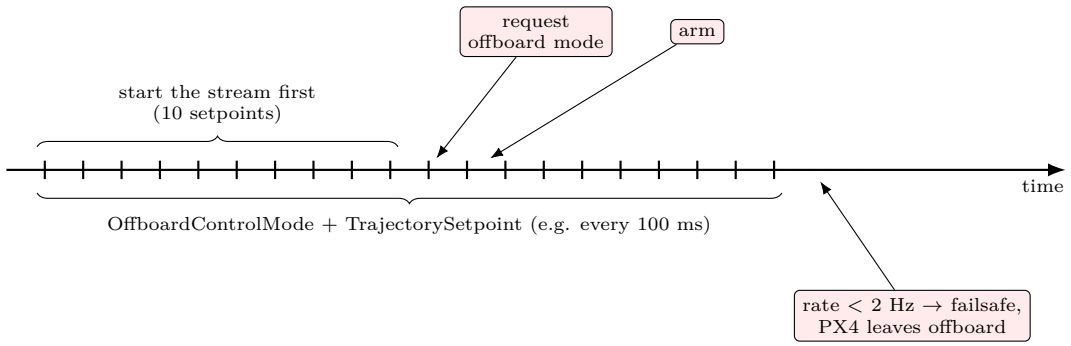


Figure 9: Offboard timeline: stream first, then request offboard mode and arm. If the stream drops below about 2 Hz, PX4 leaves offboard mode.

The three messages (fields checked in the `msg` folder of `px4_msgs`):

- `OffboardControlMode`: `timestamp` and the booleans `position`, `velocity`, `acceleration`, `attitude`, `body_rate`, `thrust_and_torque`, `direct_actuator`. For acceleration control set `acceleration = true` and the others false.
- `TrajectorySetpoint`: `timestamp`, `position[3]`, `velocity[3]`, `acceleration[3]`, `jerk[3]`, `yaw`, `yawspeed`. In NED, metres and seconds. Set every field you do not control to NaN: the message file says NaN means "the state should not be controlled". For example, to command an acceleration, set `acceleration` and leave `position` and `velocity` NaN.

- `VehicleCommand`: `command`, `param1`, `param2`, ..., `target_system`, `target_component`, `source_system`, `source_component`, `from_external`. Arm with the command `VEHICLE_CMD_COMPONENT_ARM_DISARM` (400), `param1 = 1`. Switch mode with `VEHICLE_CMD_DO_SET_MODE` (176); the official example uses `param1 = 1` and `param2 = 6` for offboard.

To see the current mode, read `VehicleStatus`: its `nav_state` equals `NAVIGATION_STATE_OFFBOARD` (14) and `arming_state` equals `ARMING_STATE_ARMED` (2) when all is well.

5.3 The uXRCE-DDS bridge

PX4 talks to ROS 2 through the Micro XRCE-DDS protocol. A *client* runs inside PX4 and an *agent* runs on the companion computer (or the PC in simulation). The agent exposes the PX4 topics as ROS 2 topics under the prefix `/fmu/`: PX4 output under `/fmu/out/`, input under `/fmu/in/`. Start it for the simulator:

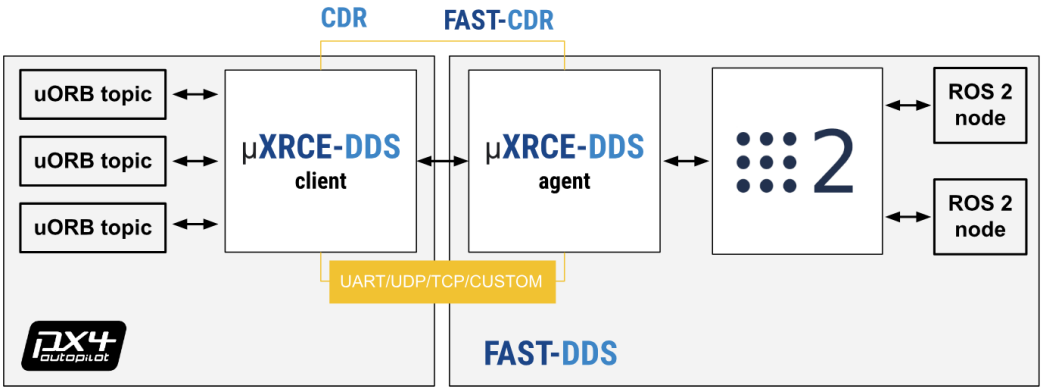


Figure 10: The uXRCE-DDS bridge: the client in PX4 talks to the agent, which exposes uORB topics as ROS 2 topics. Image: PX4 User Guide, CC BY 4.0.

```
MicroXRCEAgent udp4 -p 8888
```

The PX4 ROS 2 guide uses this command. On the real drone the link is a serial line. On our drone the agent runs as `MicroXRCEAgent serial --dev /dev/ttyTHS1 -b 921600`, and PX4 has `UXRCE_DDS_CFG = TELEM2` at 921600 baud. Expect names such as `/fmu/in/offboard_control_mode`, `/fmu/in/trajectory_setpoint`, `/fmu/in/vehicle_command`. Since PX4 1.16 some messages are versioned (the `MESSAGE_VERSION` constant in the `.msg` files), which may add a suffix to the topic name. Always run `ros2 topic list` and use what you see.

px4_msgs must match the firmware. The bridge sends messages as raw binary. If the field layout in your ROS 2 package differs from the one compiled into the firmware, data is misread or dropped. Pin `src/px4_msgs` to the release branch of your firmware (`release/1.17` for PX4 1.17, see Submodules). If the firmware changes, change the submodule with it, and rebuild with `colcon build --packages-select px4_msgs`.

5.4 Frames: NED/FRD versus ENU/FLU

PX4 and ROS use different frame conventions, and mixing them up is the most common source of bugs.

ROS (REP-103)	PX4

World frame	ENU: x East, y North, z Up	NED: x North, y East, z Down
Body frame	FLU: x forward, y left, z up	FRD: x forward, y right, z down

Consequences:

- Altitude is $-z$ in PX4. Flying 2 m above the ground means $z = -2$ in the `position` of `TrajectorySetpoint` and in `VehicleLocalPosition.z`. A positive z setpoint drives the drone into the floor.
- World vectors convert ENU to NED by swapping x and y and negating z: $(x_E, y_N, z_U) \mapsto (y_N, x_E, -z_U)$. The PX4 docs describe it as a rotation of $\pi/2$ about Z then π about X.
- Body vectors convert FLU to FRD (and back) by negating y and z (a rotation of π about X).
- Yaw: PX4 yaw is measured from North, positive clockwise seen from above; ROS yaw is measured from East, positive counter-clockwise. A conversion is needed here too.

The PX4 docs recommend the helper functions in `px4_ros_com` (`frame_transforms`). Decide once, in your project, in which frame each node works and convert only at the PX4 boundary. Many ROS nodes use ENU internally; the setpoint is then converted to NED just before publishing. An acceleration setpoint is a world-frame acceleration, so the same $x \leftrightarrow y$, $z \to -z$ rule applies to it.

5.5 Simulation: SITL with Gazebo

SITL (software in the loop) runs the real PX4 firmware as a program on your PC, with Gazebo (`gz`) simulating the drone and the world. You need your own PX4-Autopilot checkout in `~/PX4-Autopilot`, at tag `v1.17.0` (not part of your project repo; see Installing the other tools). The make targets from the PX4 docs are:

```
cd ~/PX4-Autopilot        # path is machine-specific
make px4_sitl gz_x500                 # plain x500 quadrotor
make px4_sitl gz_x500_depth           # x500 with a front depth camera
make px4_sitl gz_x500_lidar_2d        # x500 with a 2D LiDAR
```

These three targets are listed in the PX4 Gazebo page (main documentation). A world is chosen by appending its name to the target or with the environment variable `PX4_GZ_WORLD`. The first build takes several minutes. Simulated sensor images and point clouds reach ROS 2 through a Gazebo-ROS bridge (package `ros_gz_bridge`); the topic names depend on the model.

5.6 QGroundControl, parameters and safety

QGroundControl (QGC) is the ground-station program. It shows the vehicle state, lets you arm, change the flight mode, read and write PX4 *parameters*, and load failsafe settings. In simulation it connects automatically. Parameters are named values stored in the firmware (for example the one that enables the XRCE-DDS client). Change them in QGC under *Vehicle Setup -> Parameters*. Some need a reboot. Never change a parameter on the real drone without knowing what it does.

Safety in the flight cage. Follow every rule below on every flight.

- Fly in the cage with a safety pilot holding the radio transmitter. Be able to leave offboard mode with one switch and take manual control.
- Know the **kill switch**: it stops the motors at once. The drone falls. Use it only when the alternative is worse. Find its channel in QGC: the parameter `RC_MAP_KILL_SW` shows it. Ask the safety pilot which physical switch that is.

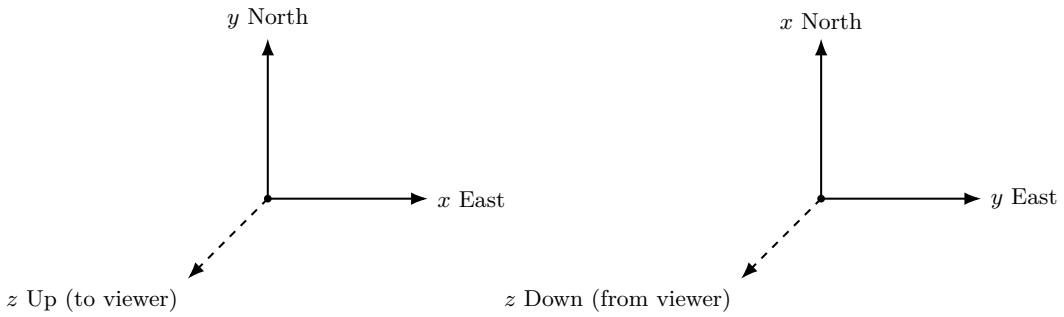


Figure 11: World frames seen from above. In ENU the third axis points up; in NED it points down. The north and east axes swap places. Body frames are FLU (ROS: forward, left, up) and FRD (PX4: forward, right, down).

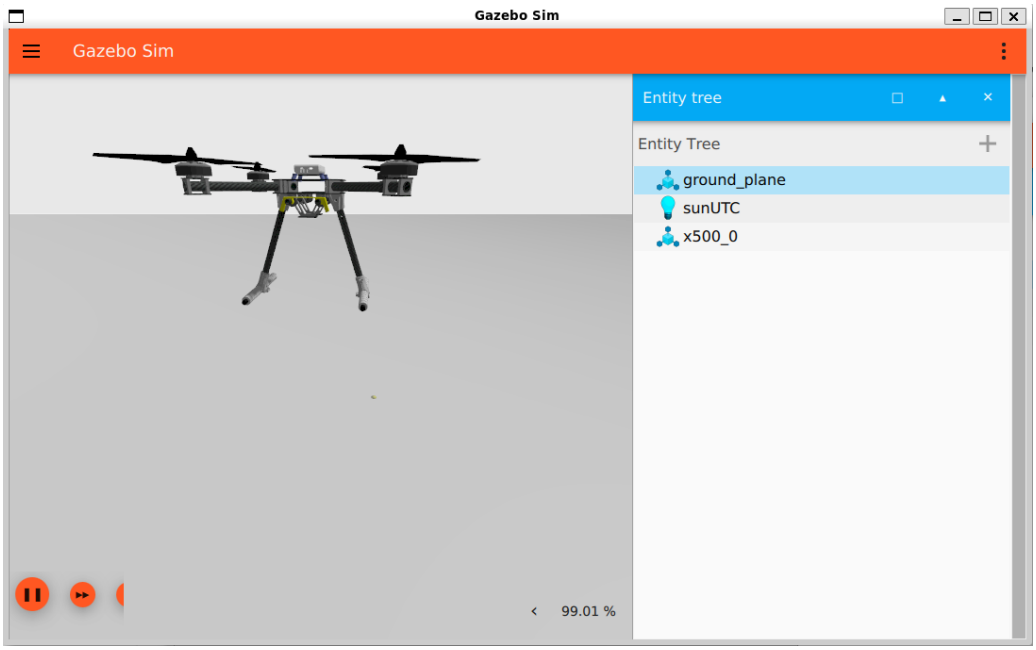


Figure 12: Gazebo with the x500 quadrotor, as started by `make px4_sitl gz_x500`. Image: PX4 User Guide, CC BY 4.0.

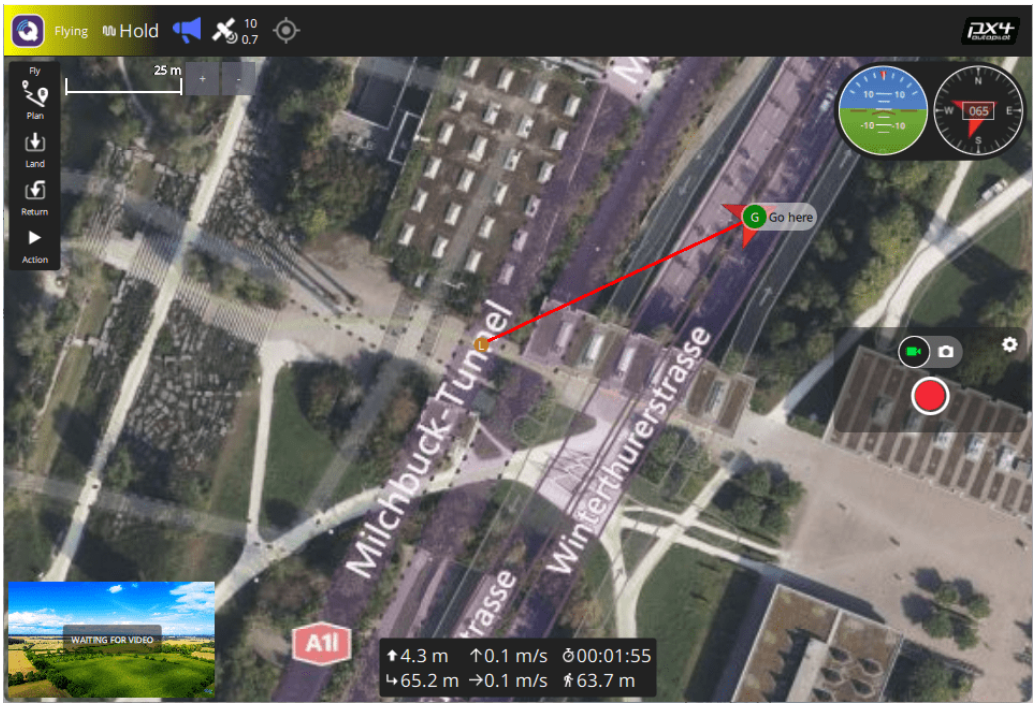


Figure 13: QGroundControl fly view: map, flight mode, attitude and arming. Image: PX4 User Guide, CC BY 4.0.

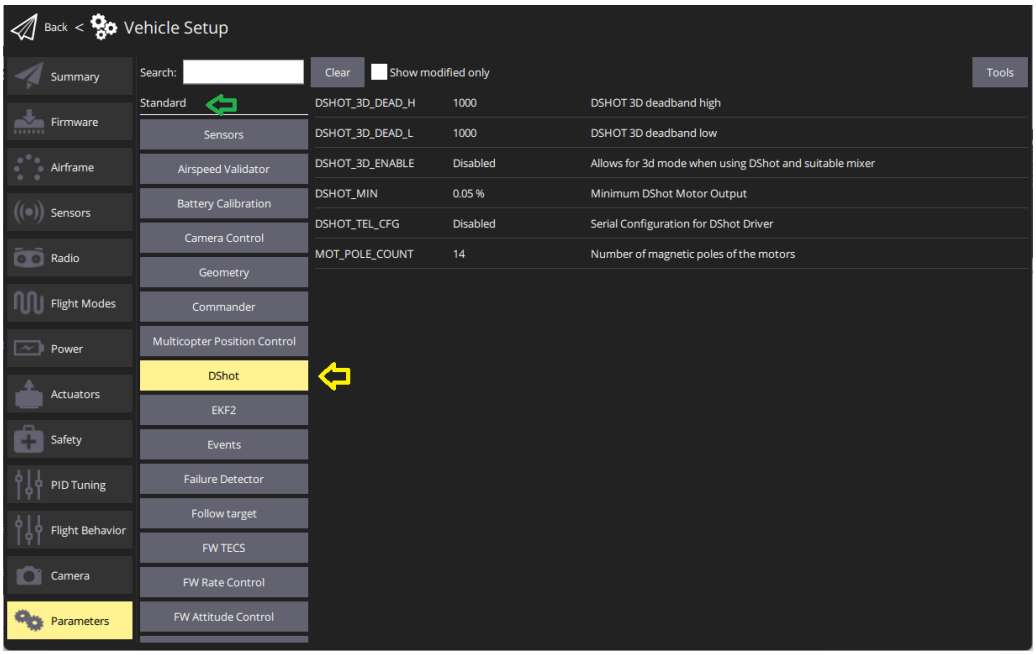


Figure 14: QGroundControl parameter editor (*Vehicle Setup -> Parameters*). Image: PX4 User Guide, CC BY 4.0.

- Test everything in simulation first. Then fly low and slow, with props on only for a planned flight. Remove propellers when working on the bench.
- If the setpoint stream stops, PX4 leaves offboard mode (see above). On our drone it then lands (PX4 parameter `COM_OBL_RC_ACT`).

6 Isaac Sim

Isaac Sim is NVIDIA's robot simulator. It renders photorealistic cameras and simulates physics on the GPU, so it is used for perception and learning work where Gazebo images are not realistic enough. It connects to ROS 2 through its own ROS 2 bridge.

It needs much more hardware than Gazebo. The current NVIDIA requirements are Ubuntu 24.04 (22.04 is deprecated), an NVIDIA RTX GPU (minimum GeForce RTX 4080, 16 GB VRAM) and 32 GB RAM. GPUs without RT cores do not work. Check the requirements page for the version you install.

> **Note**
>
> Do not install Isaac Sim on your own. Your supervisor tells you which version to use, on which machine, and which parts (ROS 2 bridge, Isaac Lab, PX4 SITL) to check out first.

7 How it fits together

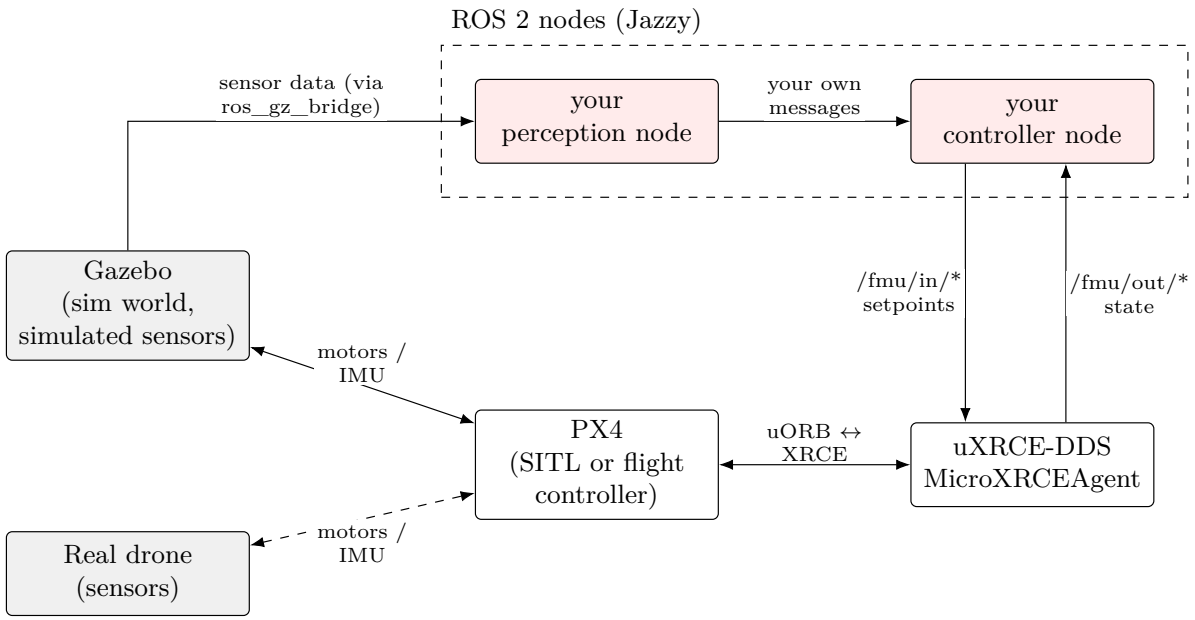


Figure 15: Data flow. Grey: the simulator or hardware. White: PX4 and its bridge. Red: your ROS 2 nodes. On the real drone, sensors reach your perception node through their own ROS 2 driver (an Intel RealSense with `realsense2_camera`), in place of the Gazebo bridge.

The sensor produces data. Your perception node turns it into something the controller can use. Your controller node reads the vehicle state from PX4 (through the bridge), computes a command, and publishes an `OffboardControlMode` and a `TrajectorySetpoint`, as in PX4, and PX4 flies it. In simulation, Gazebo plays the part of the real world, and your nodes run the same way.

7.1 First-day checklist

Run these one at a time and read the output. Paths in angle brackets are yours.

- ☐ Check the system: `lsb_release -a` (expect Ubuntu 24.04) and `ls /opt/ros` (expect `jazzy`). No `jazzy`? Install it first (Installing ROS 2 Jazzy).
- ☐ Get GitHub access and an SSH key. Test: `ssh -T git@github.com`.
- ☐ Clone with submodules: `git clone --recursive git@github.com:<owner>/<repo>.git`
- ☐ Check the submodules: `cd <repo> && git submodule status`. No line may start with `-`. If one does: `git submodule update --init --recursive`.
- ☐ Source ROS and build the messages: `source /opt/ros/jazzy/setup.bash && colcon build --packages-select px4_msgs`, then `source install/setup.bash`.
- ☐ Check a message: `ros2 interface show px4_msgs/msg/OffboardControlMode`.
- ☐ Read the project README and the open issues on GitHub.
- ☐ Set up PX4 SITL: clone PX4-Autopilot at tag `v1.17.0` into `~/PX4-Autopilot` (Installing the other tools), then in terminal 1: `make px4_sitl gz_x500`.
- ☐ Terminal 2: `MicroXRCEAgent udp4 -p 8888`.
- ☐ Terminal 3: source ROS and the workspace, then `ros2 topic list`. You should see topics under `/fmu/`. Check that one publishes: `ros2 topic hz <topic>`.
- ☐ Open QGroundControl and check that it shows the simulated vehicle.
- ☐ Hover the simulated drone in offboard mode with the example from the PX4 offboard control page (`docs.px4.io/main/en/ros2/offboard_control.html`). Follow that page for how to build and run it.
- ☐ Open a branch for your first task and push it: `git switch -c docs/1-first-day`.

8 Further reading

- Linux command line: ubuntu.com/tutorials/command-line-for-beginners
- Pro Git book: git-scm.com/book (chapters 1-3 and 7.11 “Submodules”)
- GitHub docs: docs.github.com
- ROS 2 Jazzy documentation and tutorials: docs.ros.org/en/jazzy
- ROS 2 Jazzy install on Ubuntu: docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html
- REP-103 (units and frame conventions): www.ros.org/reps/rep-0103.html
- PX4 user and developer guide: docs.px4.io (use the page for version 1.17)
- PX4 ROS 2 user guide and offboard example: docs.px4.io/main/en/ros2/user_guide.html, docs.px4.io/main/en/ros2/offboard_control.html
- PX4 Gazebo simulation: docs.px4.io/main/en/sim_gazebo_gz/
- Isaac Sim requirements: docs.isaacsim.omniverse.nvidia.com/latest/installation/requirements.html
- Message definitions: github.com/PX4/px4_msgs
- Micro XRCE-DDS Agent: github.com/eProsima/Micro-XRCE-DDS-Agent
- QGroundControl user guide: docs.qgroundcontrol.com
- Gazebo: gazebosim.org/docs